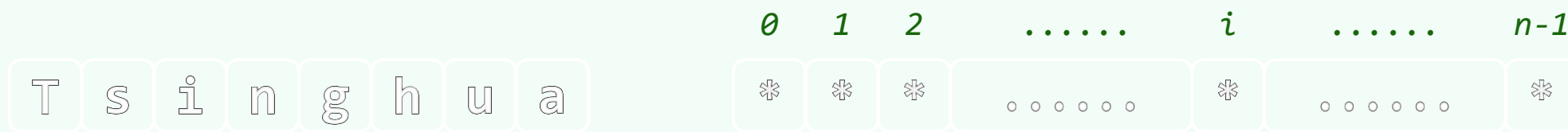


ADT

术语



$$S.substr(i, k) = S[i, i + k), \quad 0 \leq i < n, \quad 0 \leq k$$



$$S.prefix(k) = S.substr(0, k) = S[0, k), \quad 0 \leq k \leq n$$

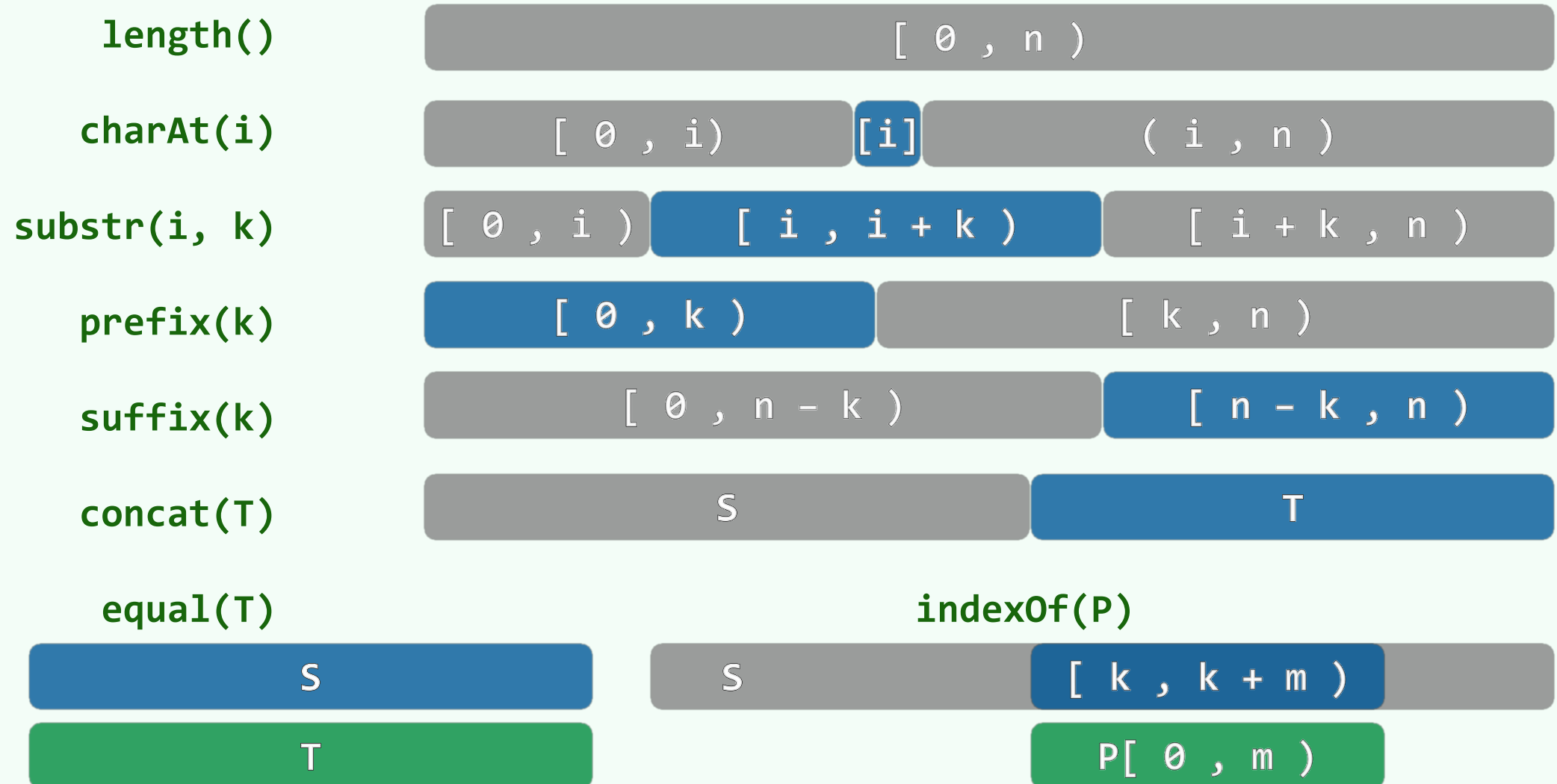


$$S.suffix(k) = S.substr(n - k, k) = S[n - k, n), \quad 0 \leq k \leq n$$



$$S.substr(i, k) = S.prefix(i + k).suffix(k) = S.suffix(n - i).prefix(k)$$

ADT



实例

- ❖ `"data structures".length() = 15` `"data structures".charAt(5) = 's'`
`"data structures".prefix(4) = "data"` `"data structures".suffix(10) = "structures"`
`"data structures".concat(" & algorithms") = "data structures & algorithms"`
`"algorithms".equal("data structures") = false`
`"data structures and algorithms".indexOf("string") = -1`
`"data structures and algorithms".indexOf("algorithm") = 20`
- ❖ **<string.h>中的对应功能:** `strlen()`、`strcpy()`、`strcat()`、`strcmp()`、`strstr()`
- ❖ 以下, 直接利用**字符数组**实现字符串, 转而重点讨论**串匹配算法**

串
模式匹配： 问题&蛮力算法

循模式访问：问题特点 + 测试方法

Text : a man with money is no match against a man on a mission

Pattern : match

random T + random P ?

$$n = |T| \quad m = |P|$$

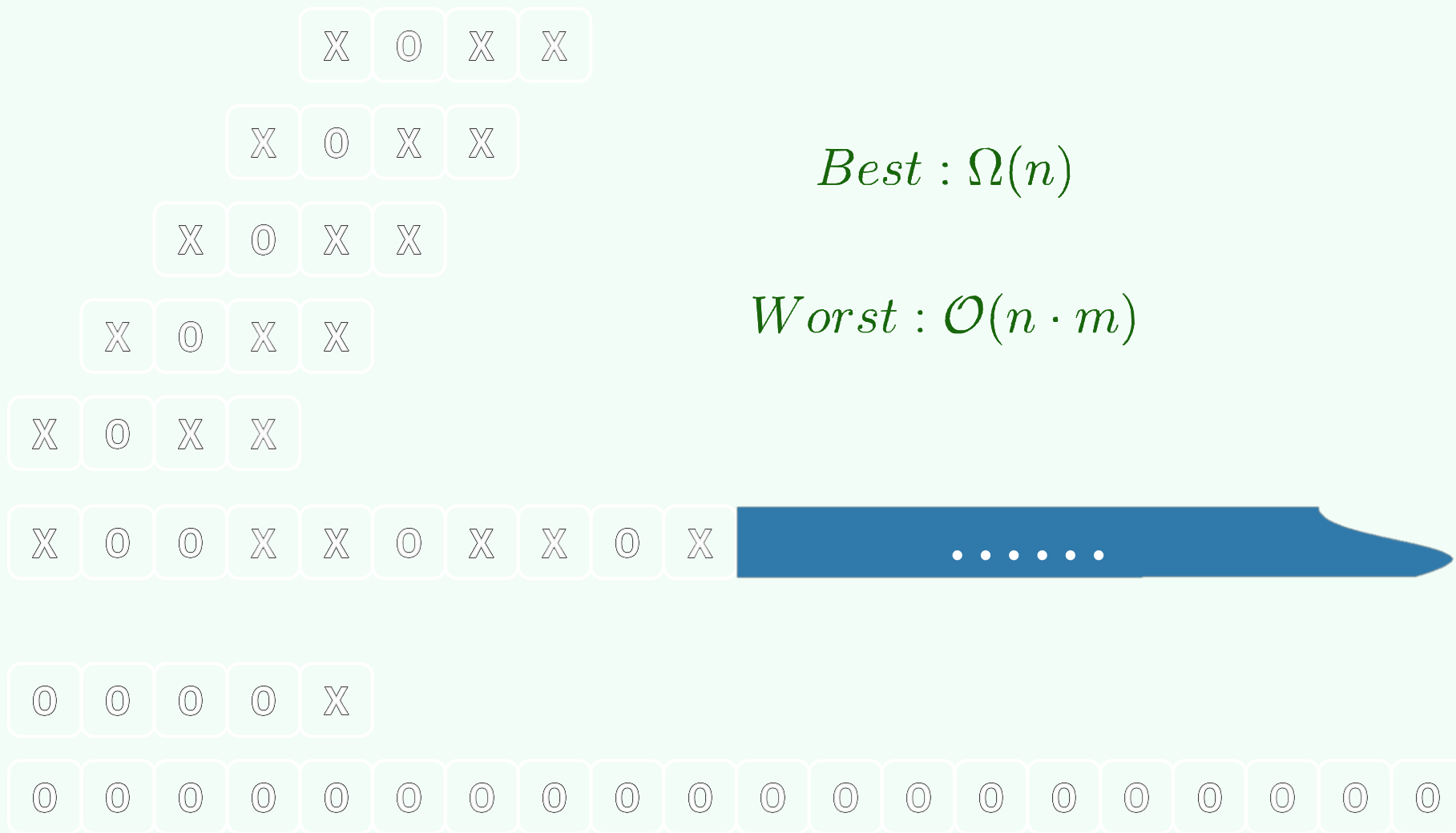
$$2 \ll m \ll n$$

random T + substring P !

$$s = |\Sigma|$$

$$Pr_{match} \approx n \cdot s^{-m} \rightarrow 0$$

蛮力策略



版本1

```
int match( char * P, char * T ) {
```

```
    size_t n = strlen(T), i = 0;
```

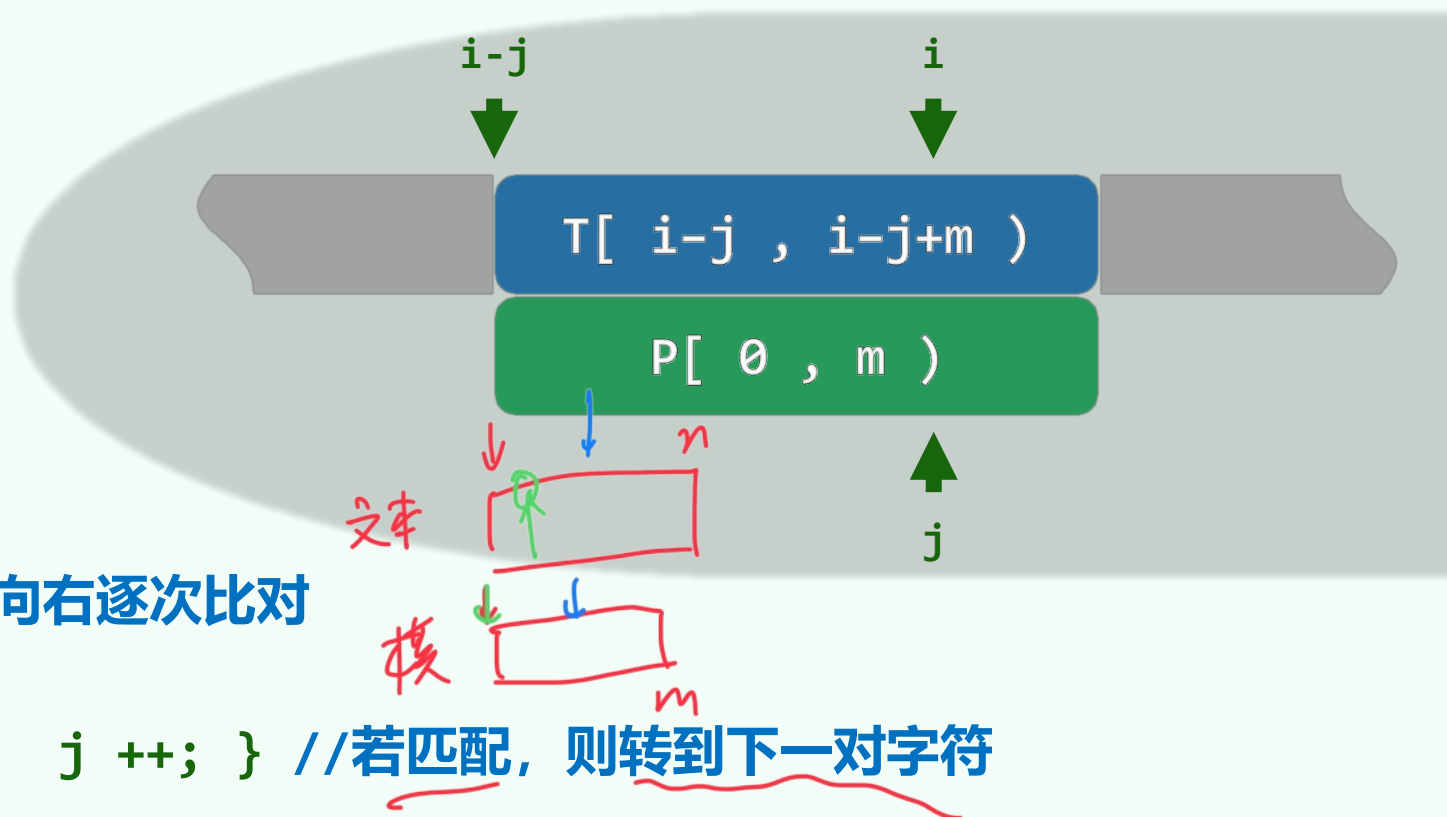
```
    size_t m = strlen(P), j = 0;
```

```
    while ( j < m && i < n ) //自左向右逐次比对
```

```
        if ( T[i] == P[j] ) { i ++; j ++; } //若匹配，则转到下一对字符
```

```
        else { i -= j-1; j = 0; } //否则，T回退、P复位
```

```
    return i-j; //最终的对齐位置：藉此足以判断匹配结果
```

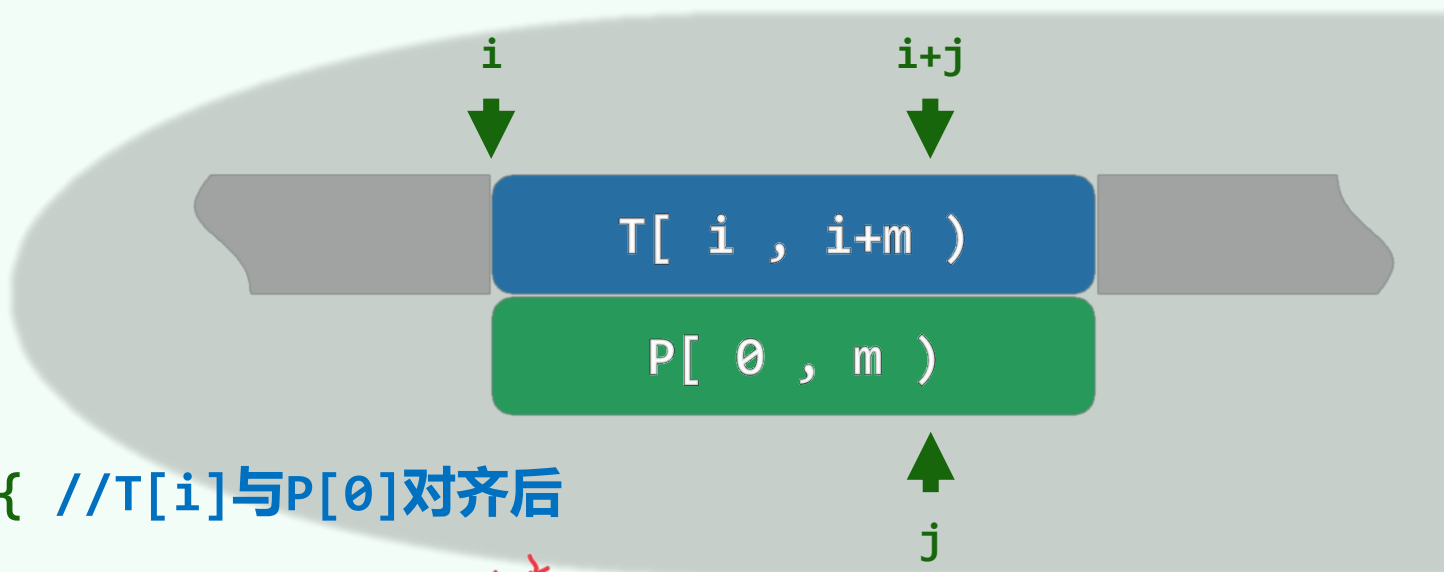


Handwritten notes:
- $i-j+1$ 回到刚刚开始的位置下-1
- 移位继续判断.

```
}
```

版本2

```
int match( char * P, char * T ) {  
    size_t n = strlen(T), i = 0;  
    size_t m = strlen(P), j;  
    for ( i = 0; i < n-m+1; i ++ ) { //T[i]与P[0]对齐后  
        for ( j = 0; j < m; j ++ ) //逐次比对  
            if ( T[i+j] != P[j] ) break; //失配, 转下一对齐位置  
        if ( m <= j ) break; //完全匹配 (Python: 可以写得更简洁、优美)  
    }  
    return i; //最终的对齐位置: 藉此足以判断匹配结果  
}
```

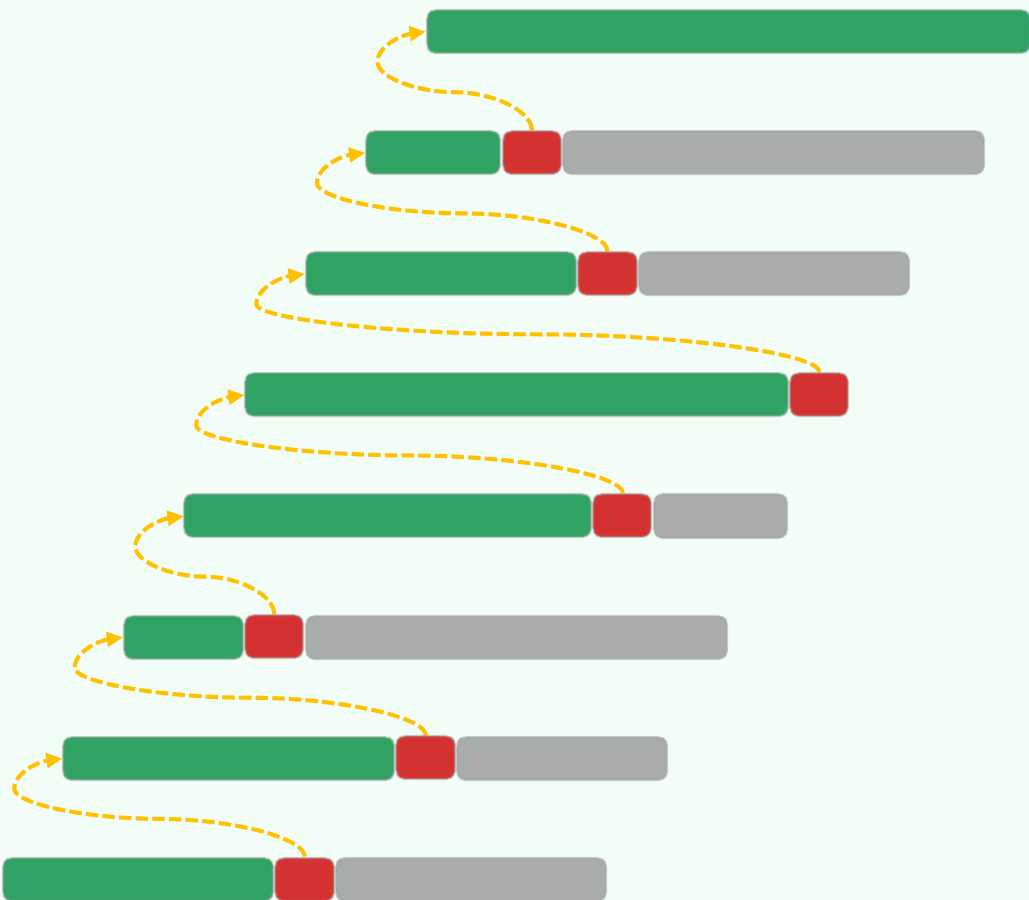


只改变j

串

KMP算法： 记忆法

低效 ~ 局部匹配



Best : $\Omega(n)$

Worst : $O(n \cdot m)$

不变性：“超强大脑”玩“记忆翻牌消消看”

❖ 在任一时刻，都有

$$T[i-j, i) == P[0, j)$$

❖ 亦即，我们业已掌握 $T[i-j, i)$ 的**所有信息**

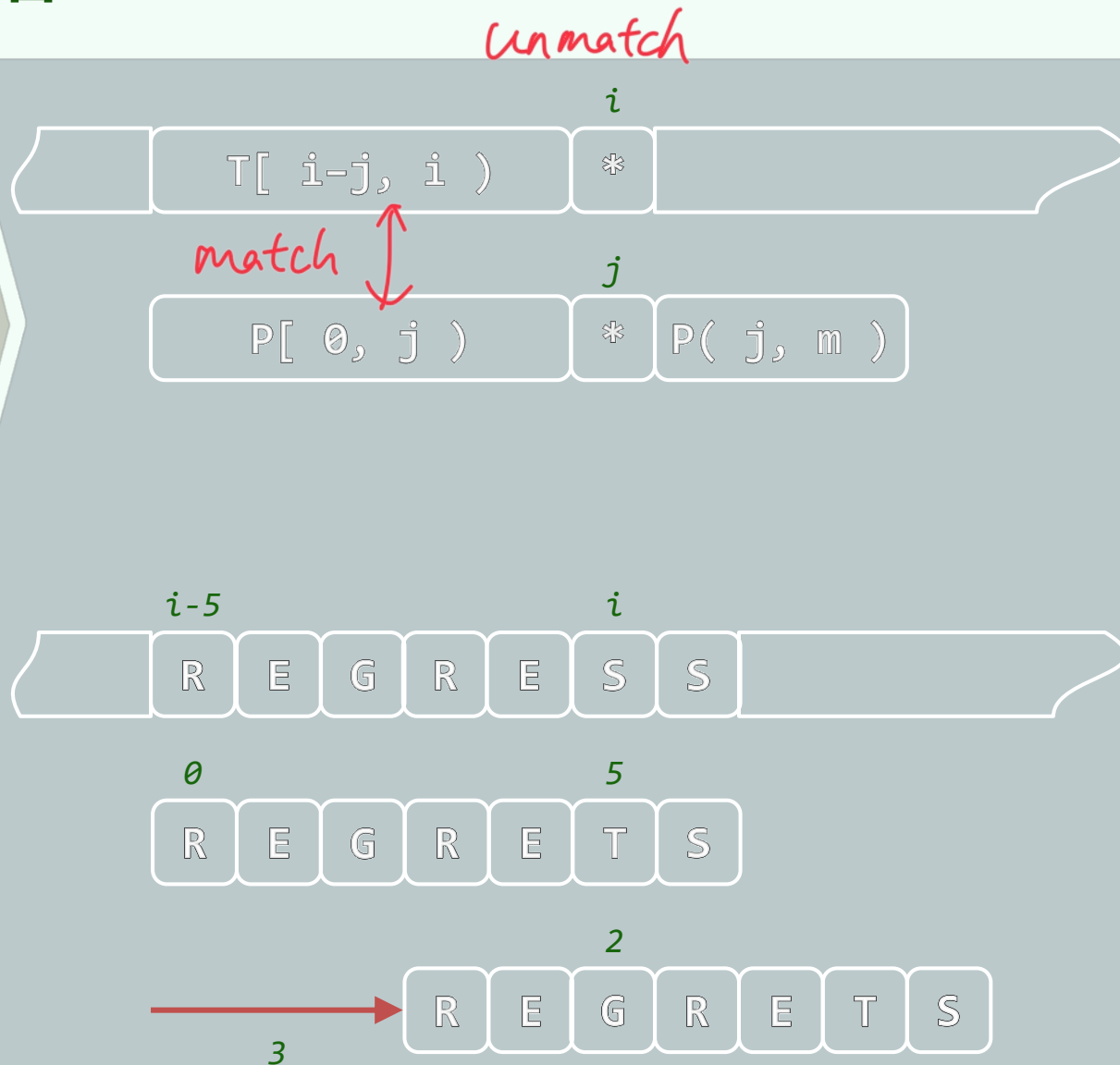
既如此...

❖ 一旦失败，我们就应已知

哪些位置**值得/不必**对齐

而且...在下一轮比对中...

❖ $T[i-j', i)$ 可**径直接受**，而**不必再做比对**



过目不忘？凡事预则立，不预则废！

❖ 如此， i 将永远不必回退！

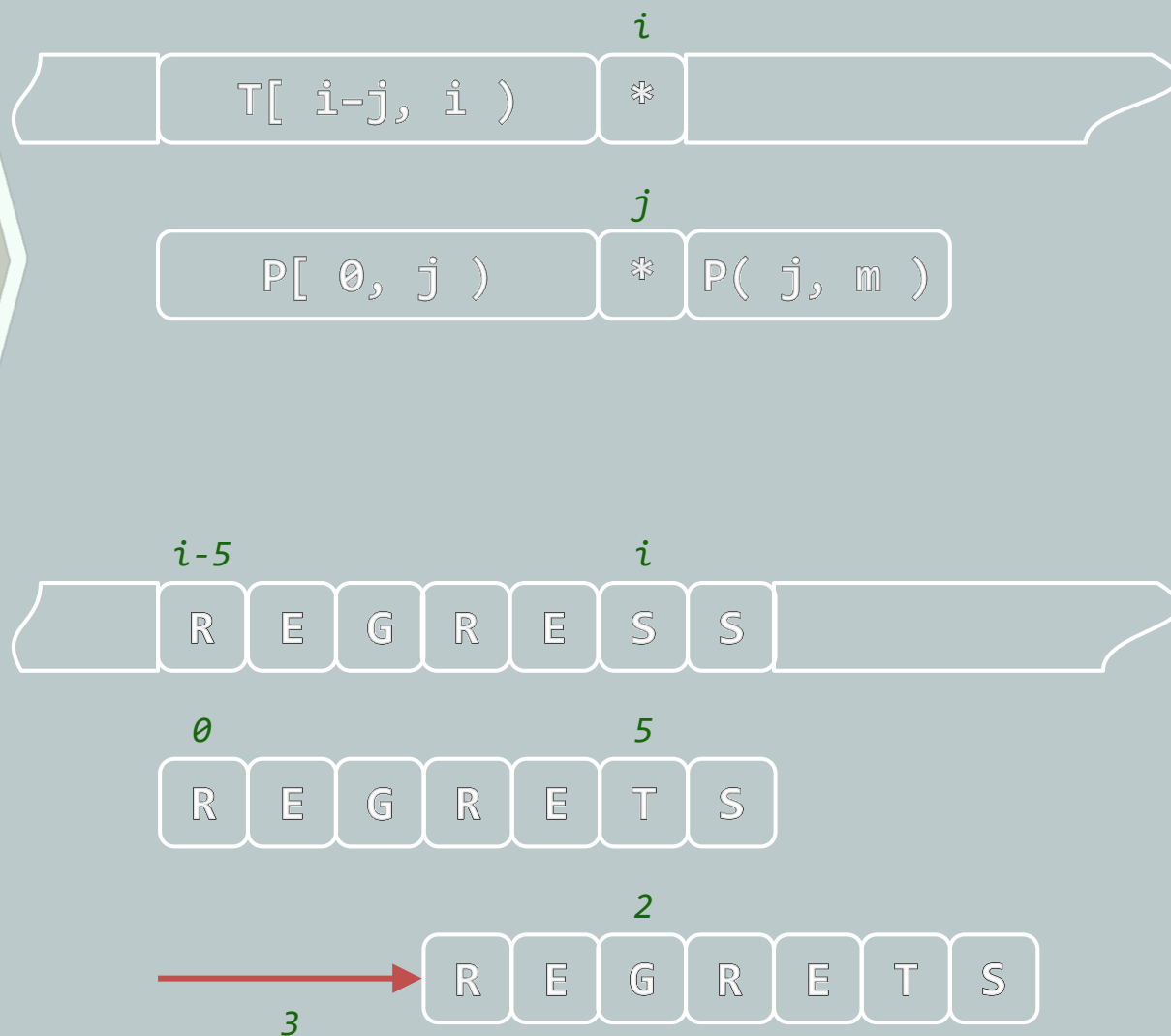
- 比对成功，则与 j 同步前进一个字符
- 否则， j 更新为某更小的 t ，并继续比对

❖ 即便是更为复杂的情况，依然可行

❖ 优化 = P可快速右移 + 避免重复比对

❖ 为确定 t ，需花费多少时间和空间？

更重要地，可否在事先就确定？



串

KMP算法： 查询表

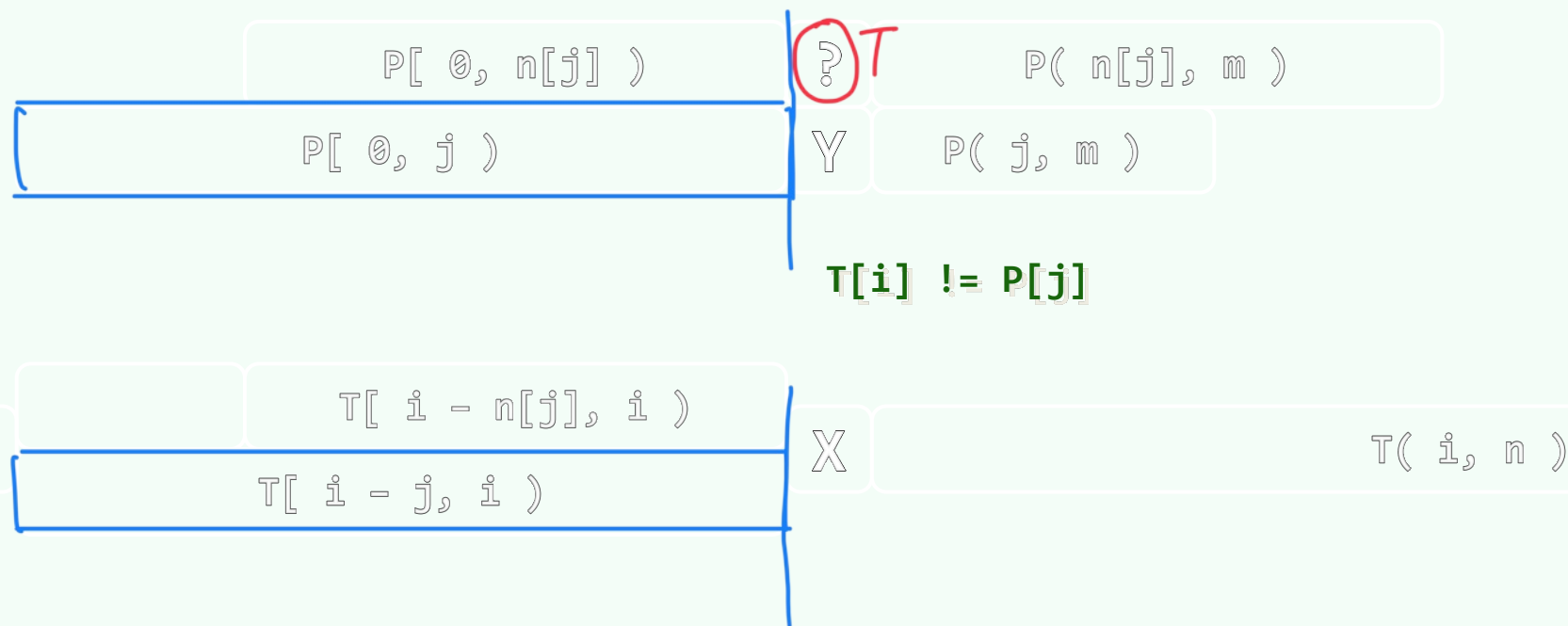
t: 不仅可以事先确定, 而且仅根据 $P[0, j) = T[i-j, i)$ 即可确定

❖ 视失败的位置 j , 无非 m 种情况

❖ 一旦在 $P[j]$ 处失配, 只需

❖ 构造查询表 $next[0, m)$, 做好预案

将 j 替换为 $next[j]$, 继续与 $T[i]$ 比对



KMP算法

```
❖ int match( char * P, char * T ) {
    int * next = buildNext(P);
    int n = (int) strlen(T), i = 0;
    int m = (int) strlen(P), j = 0;
    while ( j < m && i < n )
        if ( 0 > j || T[i] == P[j] ) {
            i ++; j ++;
        } else
            j = next[j];
    delete [] next;
    return i - j;
}
```



D. E. **K**nuth



J. H. **M**orris



V. R. **P**ratt

$P[0, n[j]]$

?

$P[n[j], m]$

$P[0, j]$

Y

$P[j, m]$

$T[i] \neq P[j]$

$T[i - n[j], i]$

X

$T[i, n]$

$T[i - j, i]$

在j=-1时. 两者也都移动
推进模式串. 文本串不动

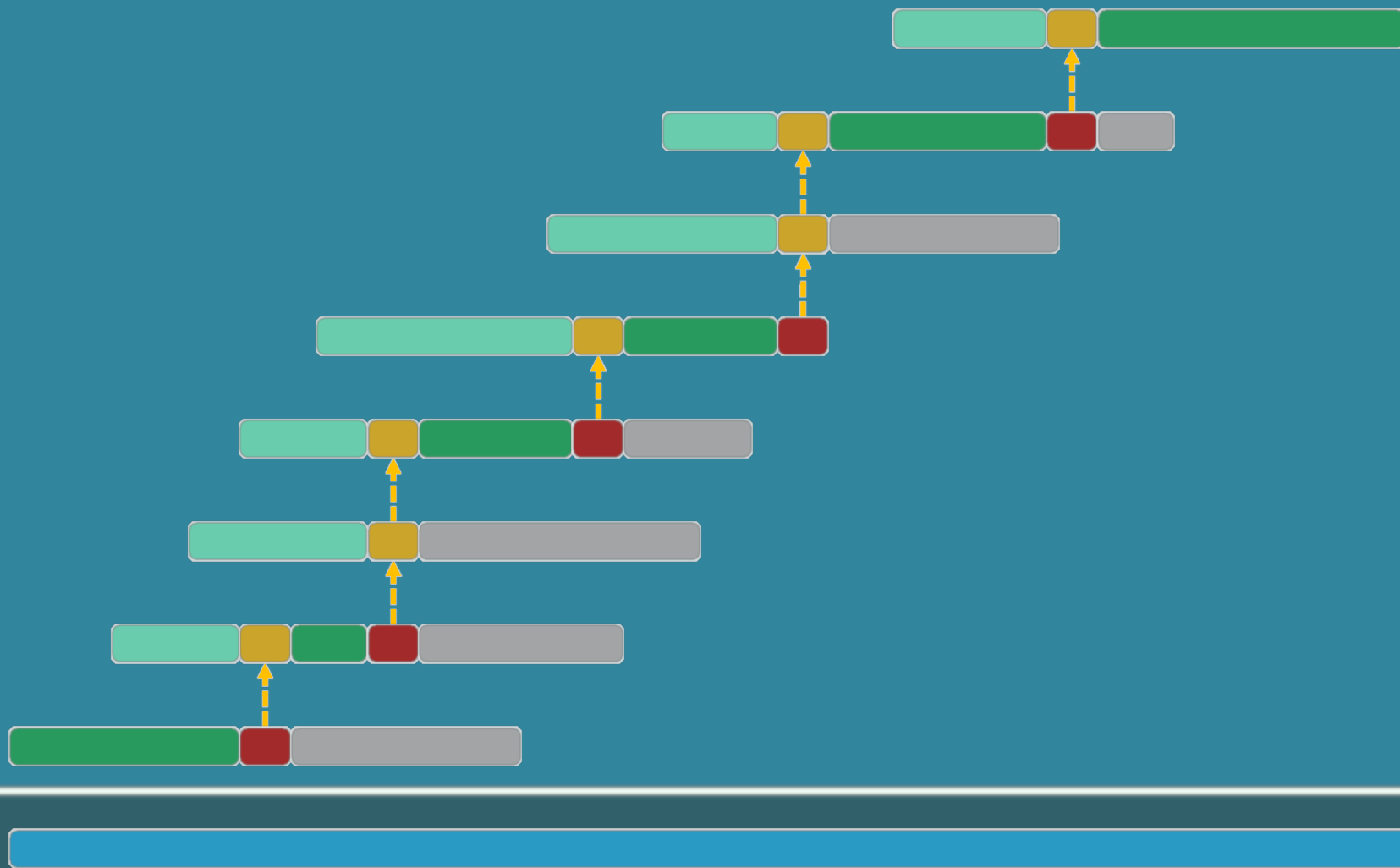
实例



	-1	0	0	0	0	1	2	3	0	0
*	C	H	I	N	C	H	I	L	L	A
-1	0	1	2	3	4	5	6	7	8	9



快速右移 + 绝不回退



串

KMP算法：理解next[]表

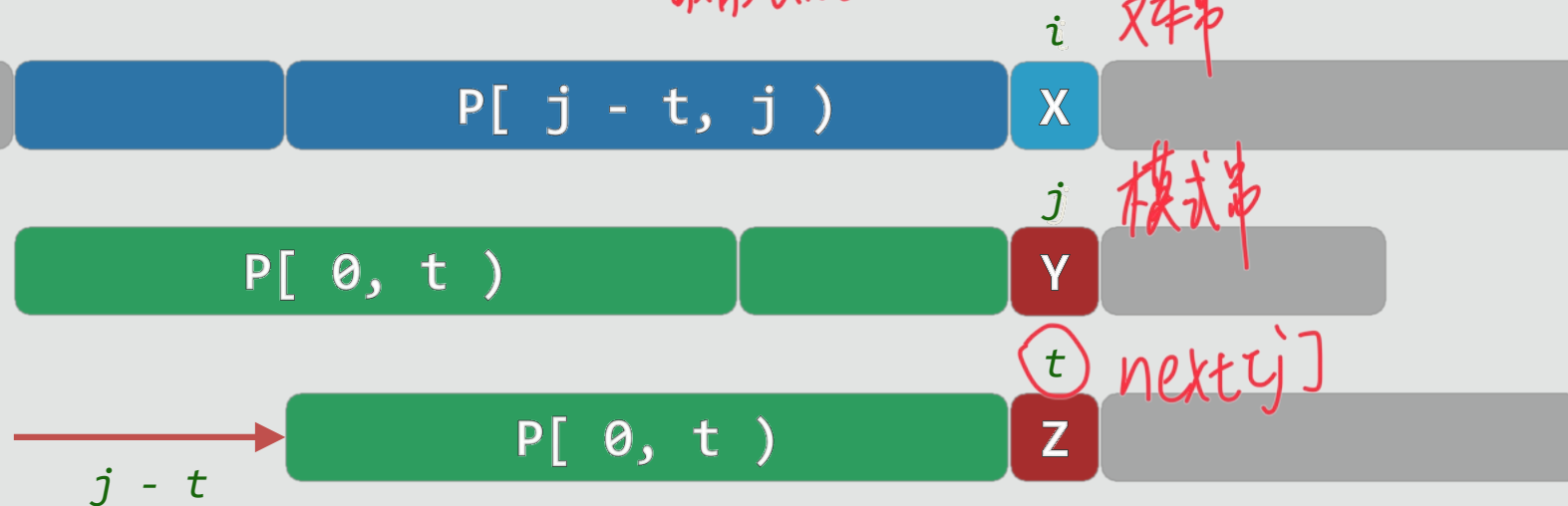
自匹配(快速右移) + 最长匹配(绝不回退)

$$\forall j \geq 1, N(P, j) = \{ 0 \leq t < j \mid P[0, t) = P[j - t, j) \}$$

$$next[j] = \max\{ N(P, j) \}$$

取最长的

j 与 i 匹配, 找 $next[j]$



$$next[0] \equiv -1$$

串

KMP算法：构造next[]表

减而治之

每一个都依赖前一个

$$next[j+1] = next[j] + 1 \quad \text{iff} \quad P[j] = P[next[j]]$$

-1 0 0 1 2 3 1 0
M A M A M M I A

-1 0 0 1
M A M A M M I A
M A M A M M I A

-1 0 0 1 2
M A M A M M I A
M A M A M M I A

-1 0 0 1 2 3
M A M A M M I A
M A M A M M I A

-1 0 0 1 2 3 1
M A M A M M I A
M A M A M M I A
M A M A M M I A
M A M A M M I A

不相等
再往前找

j j+1

P[0, j) X P(j, m)

P[0, n[j]) X P(n[j], m)

长度为 next[j] 第 next[j] 元素

P[0, n[j]) Y P(n[j], m)

P[0, n[n[j]]) ? P(n[n[j]], m)

算法实现

```
int * buildNext( char * P ) {
```

```
    size_t m = strlen(P), j = 0;
```

```
    int *N = new int[m];
```

```
    int t = N[0] = -1;
```

```
    while ( j < m - 1 )
```

```
        ( 0 > t || P[j] == P[t] ) ?
```

```
            N[ ++j ] = ++t : t = N[t];
```

```
    return N;
```

```
}
```

j $j+1$

* $P(-1, m) = P(0, m)$

$P(0, n[n[n[j]])$ X $P(n[n[n[j]]], m)$

$P(0, n[n[j]])$ Z $P(n[n[j]], m)$

$P(0, n[j])$ Y $P(n[j], m)$

$while (j < m - 1)$

$\{ if (t > 0 || P[t] == P[j])$

$\{ t++; j++; next[j] = t; \}$

$P(0, j)$ X $P(j, m)$

$else \{ t = next[t]; \}$

失配继续找

主串

模式串

串

KMP算法： 时间复杂度

串
KMP算法：再改进

反例

❖ 在 $T[3]$ 处

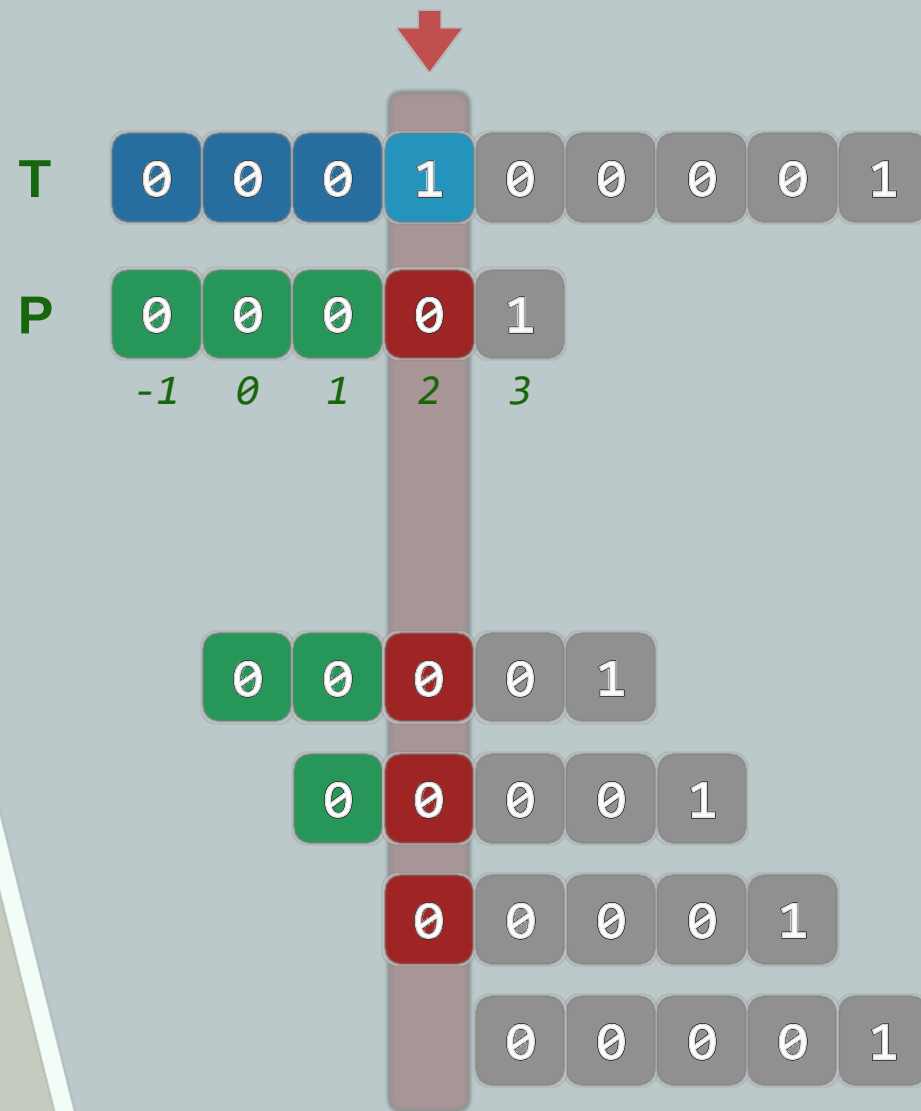
又：与 $P[3]$ 比对，失败

双：与 $P[2] = P[\text{next}[3]]$ 比对，失败

姦：与 $P[1] = P[\text{next}[2]]$ 比对，失败

姦：与 $P[0] = P[\text{next}[1]]$ 比对，失败

最终，才前进到 $T[4]$



根源

❖ 无需T串，即可在事先确定：

P[3] =

P[2] =

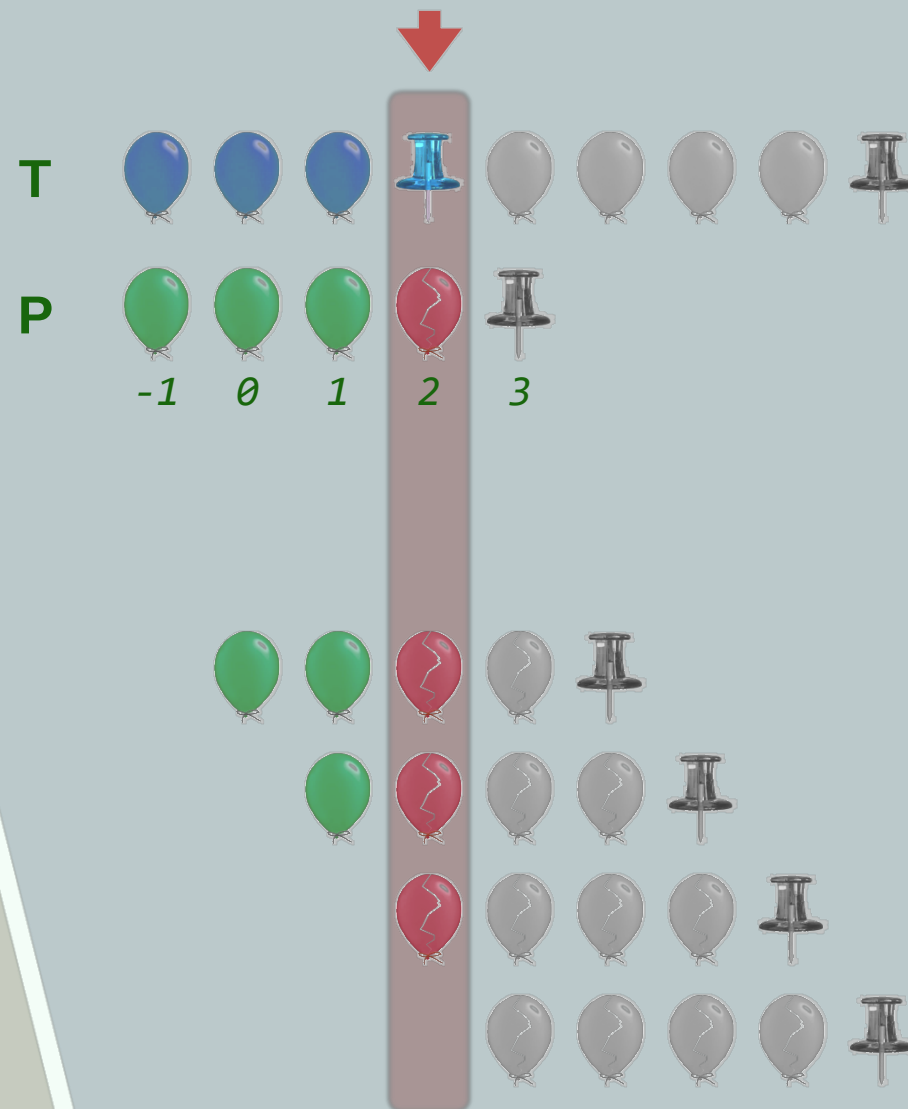
P[1] =

$$P[\emptyset] = \emptyset$$

既然如此...

❖ 在发现 $T[3] \neq P[3]$ 之后，为何还要一错再错？

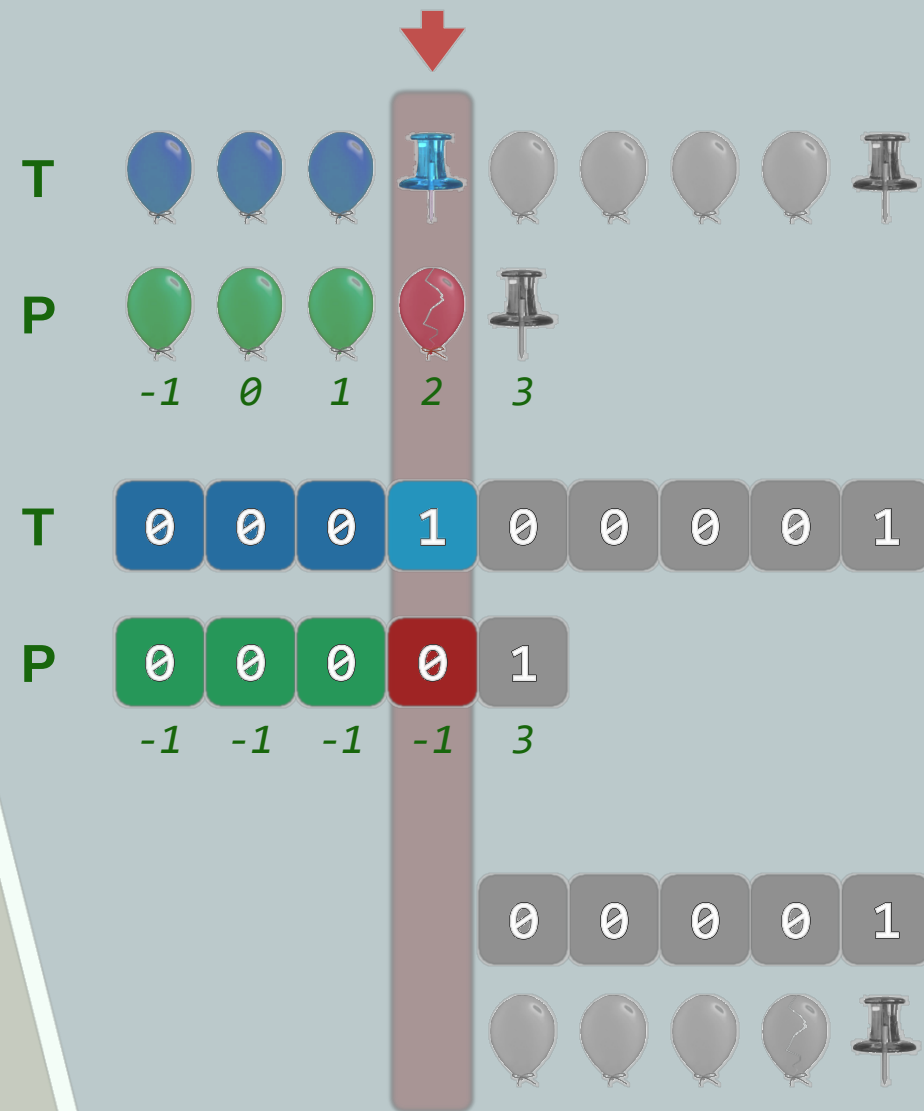
❖事实上，后三次比对本来都是可以避免的！



改进

```
int * buildNext( char * P ) {  
    size_t m = strlen(P), j = 0;  
    int * N = new int[m];  
    int t = N[0] = -1;  
    while ( j < m - 1 )  
        if ( 0 > t || P[j] == P[t] ) {  
            j ++; t ++;  
            N[j] = ( P[j] != P[t] ? t : N[t] );  
        } else  
            t = N[t];  
    return N;  
}
```

原 P[j++] != P[t++] 少一步判断



对比

-1 0 -1 0 -1 3 1 0
M A M A M M I A

初始化

$j = 0, t = -1$

$N[0] = -1$

$P[0] == P[-1]$

$j = 1, t = 0$

$P[1] != P[0]$

$N[1] = 0$

$P[1] != P[0]$

$t = N[0] = -1$

$P[1] == P[-1]$

$j = 2, t = 0$

$P[2] == P[0]$

$N[2] = N[0] = -1$ 不用再回溯了

$P[2] == P[0]$

$j = 3, t = 1$

$P[3] == P[1]$

$N[3] = N[1] = 0$

$P[3] == P[1]$

$j = 4, t = 2$

$P[4] == P[2]$

$N[4] = N[2] = -1$



初始化

$j = 0, t = -1$

$N[0] = -1$

$P[0] == P[-1]$

$j = 1, t = 0$

$N[1] = 0$

$P[1] != P[0]$

$t = N[0] = -1$

$P[1] == P[-1]$

$j = 2, t = 0$

$N[2] = 0$

$P[2] == P[0]$

$j = 3, t = 1$

$N[3] = 1$

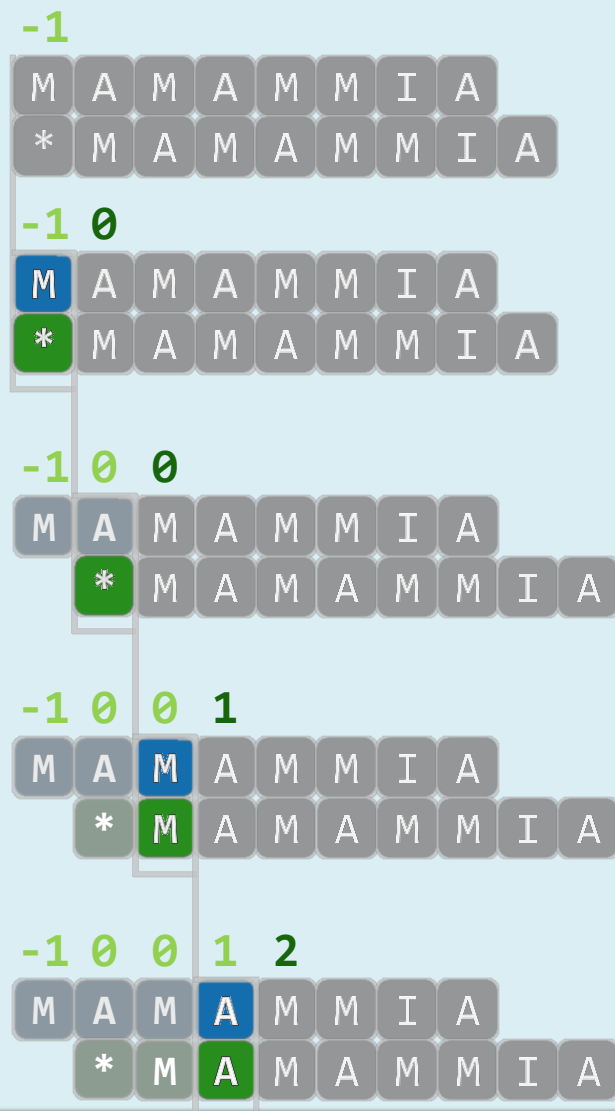
$P[3] == P[1]$

$j = 4, t = 2$

$N[4] = 2$

else后回溯

-1 0 0 1 2 3 1 0
M A M A M M I A



对比

-1 0 -1 0 -1 3 1 0
M A M A M M I A

P[6] != P[1]
t = N[1] = 0
P[6] != P[0]
t = N[0] = -1
j = 7, t = 0
P[7] != P[0]
N[7] = 0

-1 0 -1 0 -1 3 1 0
M A M A M M I A

M A M A M M I A
* M A M A M M I A

-1 0 -1 0 -1 3 1
M A M A M M I A

* M A M A M M I A
* M A M A M M I A

-1 0 -1 0 -1 3
M A M A M M I A

* M A M A M M I A

-1 0 -1 0 -1
M A M A M M I A

* M A M A M M I A

P[5] != P[3]
t = N[3] = 0
P[5] == P[0]
j = 6, t = 1
P[6] != P[1]
N[6] = 1

P[4] == P[2]
j = 5, t = 3
P[5] != P[3]
N[5] = 3

P[3] == P[1]
j = 4, t = 2
P[4] == P[2]
N[4] = N[2] = -1

-1 0 0 1 2 3 1 0
M A M A M M I A

P[6] != P[1]
t = N[1] = 0
P[6] != P[0]
t = N[0] = -1
j = 7, t = 0
N[7] = 0

-1 0 0 1 2 3 1 0
M A M A M M I A

M A M A M M I A
* M A M A M M I A

-1 0 0 1 2 3 1
M A M A M M I A

* M A M A M M I A
* M A M A M M I A

-1 0 0 1 2 3
M A M A M M I A

* M A M A M M I A

-1 0 0 1 2
M A M A M M I A

* M A M A M M I A

P[5] != P[3]
t = N[3] = 1
P[5] != P[1]
t = N[1] = 0
P[5] == P[0]
j = 6, t = 1
N[6] = 1

P[4] == P[2]
j = 5, t = 3
N[5] = 3

P[3] == P[1]
j = 4, t = 2
N[4] = 2